1. Page Table Management

1.1 Data Structures：list & state purpose for your new or modified item.

a. struct supplemental_page_table & struct suppPage

Add supplemental_page_table to vm/page.h: internally it is a hash table that serves as an SPT for each thread. Purpose: Quickly find, insert, and delete Supplemental Page Table Entries (SPTEs) for each user thread individuallySupplemental Page Table Entries (SPTEs) for each user thread to record the status of the supplemental page independently.

Add suppPage: as a SPT (Supplementary Page Table) element or node significant field:

- void *va： The virtual address of the user to which this entry corresponds (the key for the SPT query).

- bool writable： Records whether the page is writable or not

- struct hash_elem hash_elem： hash table Fields used for linking

- struct frame *frame： Points to the current frame for this page (NULL if already allocated).

- enum vm_type type： The paging type of this page (e.g. VM_ANON, VM_FILE, ...)



b. In struct thread, add a new field: struct supplemental_page_table spt, each

thread has a copy of SPT.

```
#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;                  /* Page directory. */
    // Teresa
    struct thread *parent;
    struct list children;               /* List of child processes created by this thread. */
    struct child *child_info;           /* Point to the structure of this thread in parent's child list.*/

    int st_exit;
    bool child_loaded;
    struct semaphore sema_wait;

    struct list files;
    struct file *exec_file;             /* The executable file this thread is running. */
    int file_fd;

    struct supplemental_page_table *spt;

#endif
```

1.2 Algorithms:

a. SPT Lookup / Update: how your code accesses and updates SPT entries for a given page.

(1) SPT Lookup: Use SPT (hash table) to quickly lookup whether a specific virtual address has been registered (spt_find_page)

```
/* Find suppPage in SPT by virtual address, return NULL if not found */
struct suppPage *spt_find_page(struct supplemental_page_table *spt, void *va) {
    struct suppPage temp;
    temp.va = pg_round_down(va); // 保證頁對齊
    struct hash_elem *e = hash_find(&spt->spt_hash, &temp.hash_elem);
    if (e == NULL) return NULL;
    return hash_entry(e, struct suppPage, hash_elem);
}
```

(2) SPT Update: When adding a new page, call vm_alloc_page() to create a suppPage and insert it into SPT (spt_insert_page).When delete a page, call spt_remove_page() to create a suppPage and insert it into SPT (spt_insert_page).

```
/* Insert a suppPage to SPT; return true on success, false if va exists */
bool spt_insert_page(struct supplemental_page_table *spt, struct suppPage *page) {
    struct hash_elem *result = hash_insert(&spt->spt_hash, &page->hash_elem);
    return result == NULL; // NULL means insert succeeded
}

bool
vm_alloc_page(enum vm_type type, void *upage, bool writable) {
    struct supplemental_page_table *spt = &thread_current()->spt;
    upage = pg_round_down(upage);

    // 1. 查重
    if (spt_find_page(spt, upage) != NULL)
        return false;

    // 2. 建一個 suppPage
    struct suppPage *page = malloc(sizeof(struct suppPage));
    if (page == NULL)
        return false;
```

```
    page->va = upage;
    page->writable = writable;
    page->frame = NULL;
    // page->type = type;  // 如果你 struct suppPage 有 type，記得加

    // 3. 插入
    if (!spt_insert_page(spt, page)) {
        free(page);
        return false;
    }
    return true;
}
```

The page declares the process:

1. User needs new page, call vm_alloc_page to register in SPT
2. If a page fault occurs, the handler queries the SPT.
3. If the SPT has the page, call vm_do_claim_page to allocate the frame and set up the page mapping vm_do_claim_page:

```
// 假設 page->frame 欄位直接存 kva，進階可以用 struct frame
bool vm_do_claim_page(struct suppPage *page) {
    // 1. 分配一塊 user frame
    void *kva = palloc_get_page(PAL_USER);
    if (kva == NULL)
        return false;

    // 2. 建立 page <-> frame 關聯
    page->frame = kva; // 若你 struct frame，這行要修正

    // 3. 設定到使用者的 page table
    struct thread *curr = thread_current();
    if (!pagedir_set_page(curr->pagedir, page->va, kva, page->writable)) {
        // 設定失敗要 free 掉 frame
        palloc_free_page(kva);
        page->frame = NULL;
        return false;
    }

    return true;
}
```

b. Accessed & Dirty-Bit: how you keep A/D bits consistent for aliased kernel/user mappings:

In this system implementation, the Accessed/Dirty bit is automatically maintained by the hardware.When the same physical page is mapped in the user/kernel space at the same time, these mapped A/D bits will remain the same because they all point to the same physical page. SPT does not need to synchronize these states by itself.

```
// 3. 設定到使用者的 page table
struct thread *curr = thread_current();
if (!pagedir_set_page(curr->pagedir, page->va, kva, page->writable)) {
    // 設定失敗要 free 掉 frame
    palloc_free_page(kva);
    page->frame = NULL;
    return false;
}
```

1.3 Synchronization: how races are prevented when two user processes simultaneously request

SPT initialize/destroy also per-thread in init_thread, thread_exit

```
supplemental_page_table_init(&t->spt);  // ← ← ← 加這行
#endif
```

```
supplemental_page_table_destroy(&thread_current()->spt);  // ← ← ← 加這行
#endif
```

```
#ifdef USERPROG
    /* Owned by userprog/process.c. */
    uint32_t *pagedir;                  /* Page directory. */
   // Teresa
    struct thread *parent;
    struct list children;               /* List of child processes created by this thread. */
    struct child *child_info;           /* Point to the structure of this thread in parent's child list.*/

    int st_exit;
    bool child_loaded;
    struct semaphore sema_wait;

    struct list files;
    struct file *exec_file;              /* The executable file this thread is running. */
    int file_fd;

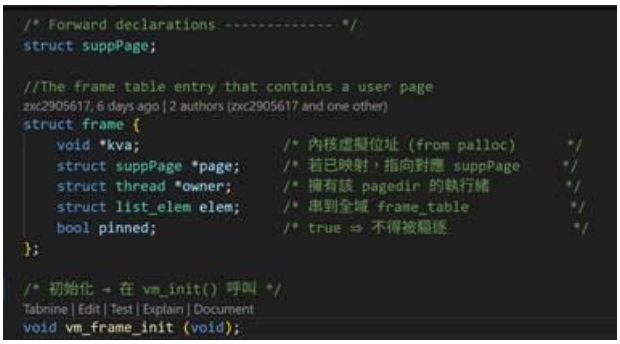
    struct supplemental_page_table *spt;
#endif
```

"Each user process/thread has its own SPT, and only it operates its own SPT. Different processes cannot access/compete for the same SPT at the same time, so there is no race condition, and no need for locks, which can be attributed to the spt field of the thread structure, and the fact that all SPT operations are performed as threads.This is evidenced by the spt field in the thread structure and the fact that all SPT operations are passed as thread _current()->spt."

2. Paging between Disk

2.1 Data Structures: As above, but only items related to paging and swap:

Frame Table: The Frame Table is used to keep track of all currently configured physical frames, and its data structure is defined as follows

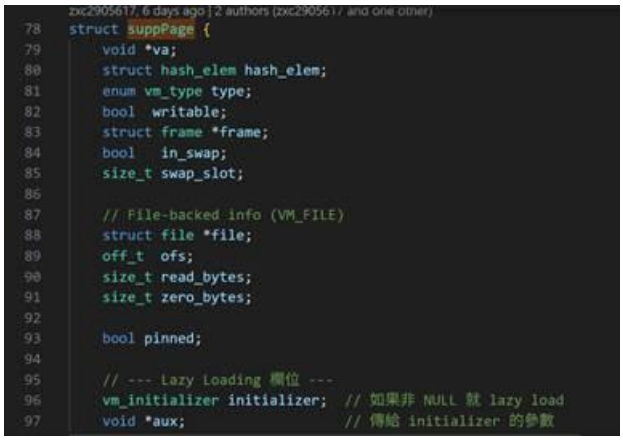


All frames are managed by the frame_table, which is a global list structure and is protected by frame_lock. pinned fields are used to prevent the page from being swapped while I/O is in progress.Whenever a page is loaded into a frame that requires memory, a request to palloc is attempted first, and if that fails, a clock algorithm is used to select a victim from the frame_table to replace.



Clock Replacement (clock_hand)：To implement the global page replacement algorithm, I design a global pointer, clock_hand, to point to the next frame to be scanned in the frame_table. clock_hand scans all the frames cyclically during the Clock algorithm, skipping the frames that have been marked as pinned (indicating that they are in use and cannot be evicted) and clearing the accessed bit of each page to provide a "second chance" mechanism.When running the Clock algorithm, clock_hand scans all frames in a loop, skipping frames that are already marked as pinned (indicating that they are in use and cannot be evicted) and clearing the accessed bit of each page to provide a "second chance"

mechanism, which is the core logic of the Clock algorithm.To complement the frame management and replacement logic, I have extended the struct suppPage structure (defined in vm/page.h) to record the state of the user's virtual pages and to support the management and determination of the following information:

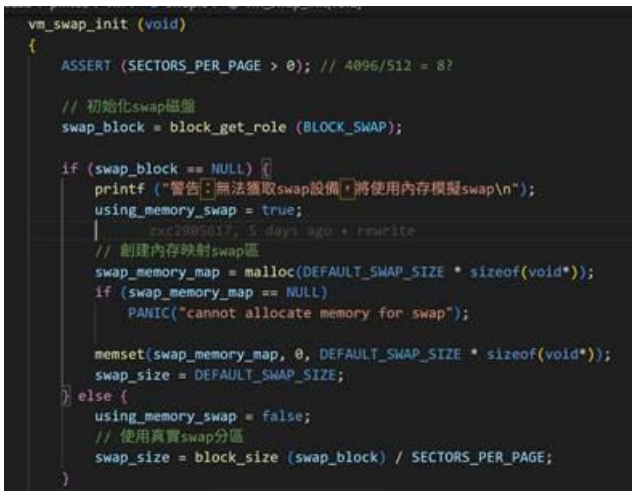


The page type, write permissions, corresponding frame, and whether or not it was swapped out of its swap slot. this information supports backfilling (from swap or file) in the event of a paging error.

Swap Table:

To support the paging out feature, I designed a Swap Table to manage the available swap areas.In practice, it consists of the following three core components:

1. swap_block: represents the real swap partition device (obtained by block_get_role(BLOCK_SWAP) provided by Pintos);
2. swap_available: Uses the bitmap data structure to manage slot utilization in the swap area;
3. swap_lock: locks all swap resource access operations (such as allocating or releasing slots) to ensure synchronization and correctness.



```
vm_swap_init (void)
{
    ASSERT (SECTORS_PER_PAGE > 0); // 4096/512 = 8?

    // 初始化swap磁盤
    swap_block = block_get_role (BLOCK_SWAP);

    if (swap_block == NULL) {
        printf ("警告：無法獲取swap設備，將使用內存模擬swap\n");
        using_memory_swap = true;
        zxc2905617, 5 days ago • rewrite
        // 創建內存映射swap區
        swap_memory_map = malloc(DEFAULT_SWAP_SIZE * sizeof(void*));
        if (swap_memory_map == NULL)
            PANIC("cannot allocate memory for swap");

        memset(swap_memory_map, 0, DEFAULT_SWAP_SIZE * sizeof(void*));
        swap_size = DEFAULT_SWAP_SIZE;
    } else {
        using_memory_swap = false;
        // 使用真實swap分區
        swap_size = block_size (swap_block) / SECTORS_PER_PAGE;
    }
```



```
// 初始化 swap_available，所有條目都為 true
swap_available = bitmap_create (swap_size);
if (swap_available == NULL)
    PANIC ("無法創建 swap_available 位圖");

bitmap_set_all (swap_available, true);
lock_init (&swap_lock);

printf ("swap區初始化完成: %zu 頁, %s\n",
        swap_size,
        using_memory_swap ? "使用內存模擬" : "使用真實swap分區");
```

Swap Write and Read:

swap_out(struct suppPage *page) writes the page into an empty swap slot (from frame->kva).Two modes are supported:

1. Use real disk (block_write)
2. Use memory emulation (memcpy to swap_memory_map[]) This function will record the swap_slot of the page and set page->in_swap = true.

swap_in(struct suppPage *page, void *kva) When a page needs to be read from the swap area back to memory, the system will load the page data from the specified swap slot into the newly allocated frame (specified kernel virtual address, kva).When the read is complete, the swap slot is released (the bitmap counterpart is reset), indicating that the location is ready to be used again.Finally, set the in_swap field of the page to false to indicate that the page is now residing in main memory and is no longer in the swap area.

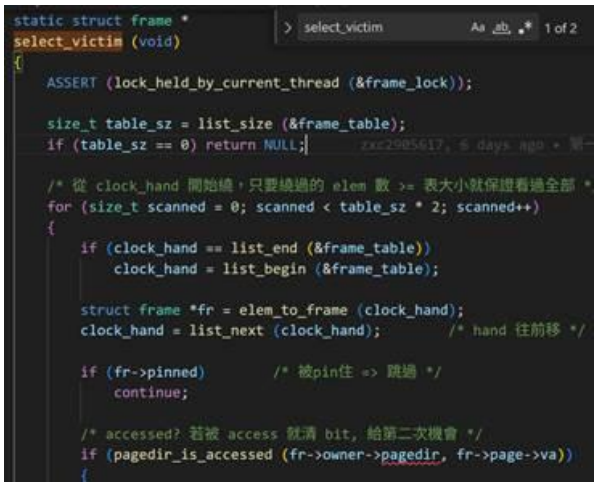
vm_swap_free(swap_index) When a swap slot is no longer needed (e.g., when a program ends or a page is released manually), the system resets the corresponding bitmap bits and

frees the used analog memory space (if allocated).To ensure secure access to swap resources, all swap-related operations are protected by swap_lock to avoid a race condition between multiple threads.In addition, during disk I/O operations, the system stores and restores interruptions to prevent interrupts from being closed at inappropriate times, thus avoiding potential deadlock or data corruption problems.

2.2 Algorithms

(a) Frame Eviction Policy:

The frame replacement algorithm uses the Clock algorithm (Second-Chance Approximation of LRU).In vm/frame.c, I have implemented select_victim() function to select suitable frame to be evicted.



Scanning range control - To prevent endless loops, I designed the scanning range to be at most twice the length of the frame_table to ensure that it is scanned completely at least once.

Clock Hand Logic - Uses the global variable clock_hand as a loop scanning indicator, if it points to list_end, it will be reset to list_begin.

Victim judgment logic - scans one frame at a time:

- If the pinned flag of the frame is true (I/O in progress), it is skipped.
- If the frame's page has been "recently used" (accessed bit = 1), the accessed bit is

cleared and skipped (giving a second chance).

- If unpinned and accessed = 0, then select victim and return immediately.

End of Scan fallback-If no driveable pages can be found after scanning (i.e., they are all pinned), then NULL is returned, which means that no frames can be driven.

```
    /* accessed? 若被 access 就清 bit，給第二次機會 */
    if (pagedir_is_accessed (fr->owner->pagedir, fr->page->va))
    {
        pagedir_set_accessed (fr->owner->pagedir, fr->page->va, false);
        continue;
    }
    /* un-pinned & accessed=0 → 正式選為 victim */
    return fr;
}
/* 全 table 皆 pinned，理論上不應發生 */
return NULL;
```

Clearing the accessed bit, i.e. "giving the page a reprieve", simulates the semantics of an LRU.If the page is not used in the next round, it will be evicted.This section ensures that the I/O page will not be mistakenly replaced to avoid memory corruption.

(b) Frame Reuse Adjustment:

When memory is low, my system calls evict_frame() to evict a frame and write its contents to the corresponding redundant storage (e.g., a swap zone).This process is implemented in vm/frame.c as follows:

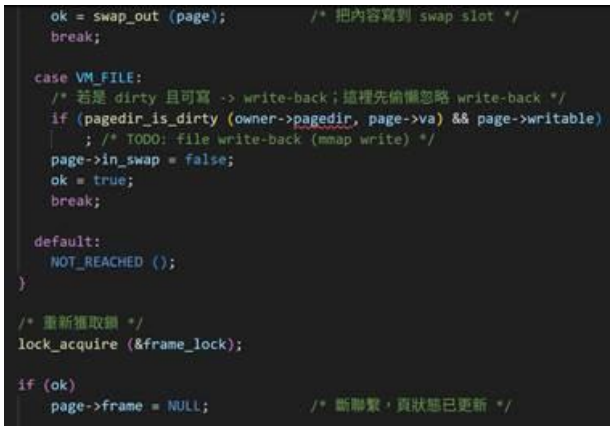
```
pintosLab > pintos > vm > C frame.c > evict_frame(frame *)
63  /* 把 victim frame 驅逐出去（swap 或寫回檔案），並返回它的 kva */
    Tabnine | Edit | Test | Fix | Explain | Document
64  static bool
65  evict_frame (struct frame *victim)
66  {
67      ASSERT (lock_held_by_current_thread (&frame_lock));
68      ASSERT (victim && victim->page);
69
70      struct suppPage *page = victim->page;
71      struct thread   *owner = victim->owner;
72
73      /* 先在 pagedir 斷開映射，避免 race */
74      pagedir_clear_page (owner->pagedir, page->va);
75
76      /* 臨時釋放鎖，以便執行可能會休眠的操作 */
77      lock_release (&frame_lock);
78
79      bool ok = false;
80      switch (page->type)
81      {
82        case VM_ANON:
83        case VM_STACK:
```

First, I remove the mapping of the page from the page table to ensure that the page can no longer be accessed, this is to avoid a race condition, if the user program accesses the page

again in the meantime, it will trigger a page fault instead of reading the wrong data.Next, the page->type determines the type of the page so that it can be processed accordingly.

VM_ANON / VM_STACK: Anonymous page or stack

The system will write the page to the swap area; after successful writing, it will record its corresponding swap location in page->swap_slot, and set in_swap to true to indicate that the page has been stored in the swap.



VM_FILE: file mapping page

Originally, it was expected that if the page was dirty and had writable permissions, a write-back operation would be performed to write the contents back to the corresponding file.However, this has not been implemented yet (noted as TODO).Therefore, for such pages, only their records in swap are cleared at the moment.

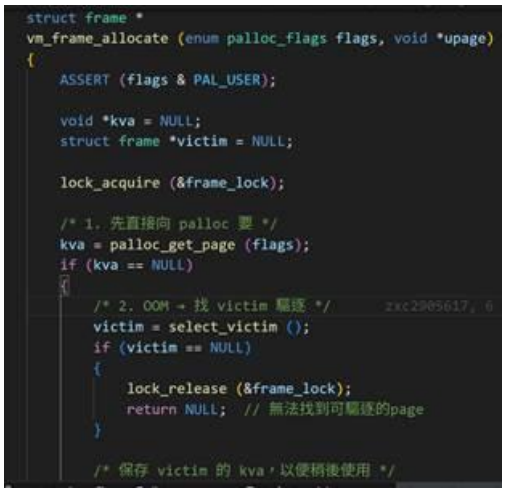
After the page is evicted, the frame is unlinked from the page and the frame resources are released (the actual memory and structure is further released in vm_frame_allocate()).

About lock handling: Since swap_out() may call malloc() or do a disk write that may cause a sleepy thread, in order to avoid entering a block while holding a lock, I release the lock before calling these operations and re-lock it after the operation is completed.This is a critical step in multi-threaded synchronization.

Supplementary vm_frame_allocate() description:

vm_frame_allocate() is the core function in all claim page allocations. It is responsible for

allocating a physical page frame and triggering a page eviction if memory is full.Its flow can be divided into three main parts:



```
struct frame *
vm_frame_allocate (enum palloc_flags flags, void *upage)
{
    ASSERT (flags & PAL_USER);

    void *kva = NULL;
    struct frame *victim = NULL;

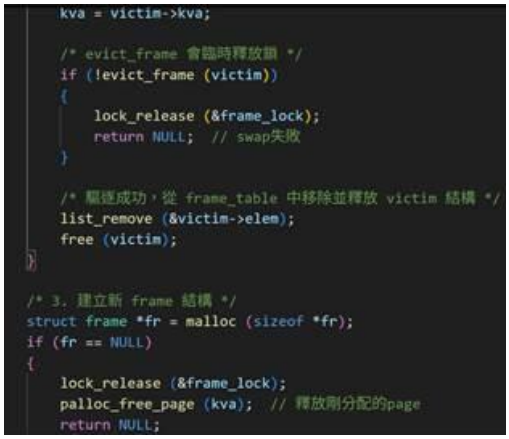
    lock_acquire (&frame_lock);

    /* 1. 先直接向 palloc 要 */
    kva = palloc_get_page (flags);
    if (kva == NULL)
    {
        /* 2. OOM → 找 victim 驅逐 */
        victim = select_victim ();
        if (victim == NULL)
        {
            lock_release (&frame_lock);
            return NULL;  // 無法找到可驅逐的page
        }

        /* 保存 victim 的 kva，以便稍後使用 */
```

1. Try to allocate directly to the palloc:
   If the system still has free physical memory, it can allocate it successfully and send back the corresponding memory address.The address allocated is a kernel virtual address that will be used to map to the user page.
2. OOM: Finding the victim frame and expelling it. If palloc_get_page() returns NULL (i.e., there is no more physical page available, i.e., OOM), select_victim() is called to try to select a page that can be evicted as the victim; if it succeeds in selecting a victim, the following steps are performed:

   (1) Store the kernel virtual address (kva) of the victim for reuse in new pages;



```
        kva = victim->kva;

        /* evict_frame 會臨時釋放鎖 */
        if (!evict_frame (victim))
        {
            lock_release (&frame_lock);
            return NULL;  // swap失敗
        }

        /* 驅逐成功，從 frame_table 中移除並釋放 victim 結構 */
        list_remove (&victim->elem);
        free (victim);
    }

    /* 3. 建立新 frame 結構 */
    struct frame *fr = malloc (sizeof *fr);
    if (fr == NULL)
    {
        lock_release (&frame_lock);
        palloc_free_page (kva);  // 釋放剛分配的page
        return NULL;
```

   (2) Call evict_frame() to evict the content corresponding to the victim from memory

(possibly writing to the swap area);

(3) Release the frame structure used by the victim;

(4) Remove the frame from frame_table.

This process is tightly integrated with the Clock algorithm and the swap-out module to form a complete and efficient page eviction mechanism.

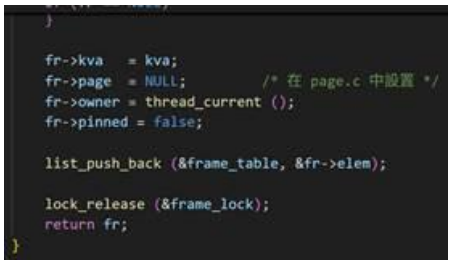
3. Create and initialize a new frame structure.

   After successfully obtaining a physical page, the relevant fields of the frame structure are initialized, including:

   - kva (kernel virtual address): the corresponding kernel virtual address;
   - owner: the thread to which the frame belongs;
   - pinned: Marks if the frame is temporarily not evictable (e.g. in I/O use).

   This is not yet bound to a specific page structure, and will be set later by logic in page.c.

   Finally, the frame is added to the frame_table to ensure that it can later participate in a page replacement or be properly released when it is no longer in use.



2.3 Synchronization

(a) VM-Wide Locking Strategy – basic design and how it avoids deadlock.

I use a global lock frame_lock to protect access to and modification of the following resources:

```c
static struct list frame_table;       /* 雙向循環 list – 保存所有 frame */
static struct lock frame_lock;        /* 保護 frame_table + clock_hand */
static struct list_elem *clock_hand;  /* 指向下一個要檢查的 frame     */
```

```c
/*
 * 初始化 frame
 * 將會在 vm_init 中調用
 */
Tabnine | Edit | Test | Explain | Document
void
vm_frame_init (void)
{
    list_init (&frame_table);
    lock_init (&frame_lock);
    clock_hand = list_end (&frame_table);
}
```

To ensure the correctness and security of frame operations in the virtual memory subsystem, all data structures and processes related to frame management are protected by a global frame_lock.

First of all, frame_table is a shared resource in the whole domain, and it is necessary to prevent multiple threads from adding or removing frames to it at the same time, so all the accesses to the frame_table need to be locked.Similarly, the clock_hand pointer used by the Clock algorithm must be protected when scanning frames to prevent race conditions from causing mis-selection or pointer confusion.

```c
lock_acquire (&frame_lock);

/* 1. 先直接向 palloc 要 */
kva = palloc_get_page (flags);
if (kva == NULL)
{
    /* 2. OOM → 找 victim 驅逐 */
    victim = select_victim ();
    if (victim == NULL)
    {
        lock_release (&frame_lock);
        return NULL;  // 無法找到可驅逐的page
    }

    /* 保存 victim 的 kva，以便稍後使用 */
    kva = victim->kva;

    /* evict_frame 會臨時釋放鎖 */
    if (!evict_frame (victim))
        lock_release (&frame_lock);
        return NULL;  // swap失敗
    }

    /* 驅逐成功，從 frame_table 中移除並釋放 victim 結構 */
    list_remove (&victim->elem);
    free (victim);
}

/* 3. 建立新 frame 結構 */
struct frame *fr = malloc (sizeof *fr);
if (fr == NULL)
{
    lock_release (&frame_lock);
    palloc_free_page (kva);  // 釋放剛分配的page
    return NULL;
}
```

The selection and eviction of the victim frame and the creation of a new frame are designed as an integrated process, with the same lock to protect its consistency.To ensure that all subsequent frame operations are correctly serialized, frame_lock is created and enabled during the initialization phase.

All functions related to frame allocation and release, such as vm_frame_allocate(),

vm_frame_free(), select_victim(), evict_frame(), etc., call lock_acquire(&frame_lock) before entering the critical section, and then call lock_acquire(&frame_lock) when finished.The lock is released after completion to ensure that the overall process is mutually exclusive and consistent in a multi-threaded environment.

(b) Cross-Process Eviction Safety

In a multi-process environment, the system allows different threads to compete for a limited number of physical memory frames, which may result in a situation where Process A must evict the frame occupied by Process B when allocating memory.To ensure the correctness and security of this cross-process eviction, the system adopts the following practices:

1. each frame maintains the owner and page indexes

In a struct frame, the owner records the thread that owns the frame, and the page records the corresponding suppPage, so that the page table and SPT of the original process can be safely manipulated when the victim frame is elected.

```
/* Forward declarations ------------- */
struct suppPage;

//The frame table entry that contains a user page
zxc2905617, last week | 2 authors (zxc2905617 and one other)
struct frame {
    void *kva;                /* 內核虛擬位址 (from palloc)      */
    struct suppPage *page;    /* 若已映射，指向對應 suppPage     */
    struct thread *owner;     /* 擁有該 pagedir 的執行緒         */
    struct list_elem elem;    /* 串到全域 frame_table            */
    bool pinned;              /* true ⇒ 不得被驅逐               */
};
```

2. Unmap the original process page table (protect the old owner)

When evicting a frame, the virtual address mapping will be lifted from the owner pagedir of the frame to prevent the owner from accessing the frame again which has been reused by another process, resulting in information leakage or system error.

```
/* 把 victim frame 驅逐出去 (swap 或寫回檔案)，並返回它的 kva */
Tabnine | Edit | Test | Fix | Explain | Document
static bool
evict_frame (struct frame *victim)
{
    ASSERT (lock_held_by_current_thread (&frame_lock));
    ASSERT (victim && victim->page);

    struct suppPage *page = victim->page;
    struct thread   *owner = victim->owner;

    /* 先在 pagedir 斷開映射，避免 race */
    pagedir_clear_page (owner->pagedir, page->va);
```

3. swap out or file synchronization for complete frame release

After unmapping, depending on the page type, a swap out or other write-back operation (e.g. file write-back if necessary) is performed.Regardless of whether the page comes from another process or not, the data will be properly stored to ensure that the owner process can safely recover from future page faults.

```
switch (page->type)
{
  case VM_ANON:
  case VM_STACK:
    ok = swap_out (page);          /* 把內容寫到 swap slot */
    break;

  case VM_FILE:
    /* 若是 dirty 且可寫 -> write-back；這裡先偷懶忽略 write-back */
    if (pagedir_is_dirty (owner->pagedir, page->va) && page->writable)
        ; /* TODO: file write-back (mmap write) */
    page->in_swap = false;
    ok = true;
    break;

  default:
    NOT_REACHED ();
}
```

4. Release frame/page association to ensure future page fault can be handled correctly.

After complete swap out, will set page->frame to NULL, completely disconnect the association, if the owner uses this page again in the future, it will trigger the page fault processing normally.Hold frame_lock throughout the process (except for swap_out, which may be dormant) to ensure that cross-process operations will not be interfered by other threads.

```
    /* 重新獲取鎖 */
    lock_acquire (&frame_lock);

    if (ok)
        page->frame = NULL;             /* 斷聯繫，頁狀態已更新 */

    return ok;
}
```

(c) I/O In-Flight Protection.

In order to prevent data inconsistency or system error caused by the page replacement logic selecting victim during I/O operations (e.g., swap-in/swap-out or file read/write) in the paging system, an I/O in-flight protection mechanism has been implemented in this system.

1. Pinned Bit is implemented in struct frame: Each frame structure (struct frame) has a bool pinned field, which is used to indicate whether the page is currently occupied by an I/O operation.

```
struct frame {
    void *kva;                  /* 內核虛擬位址 (from palloc)      */
    struct suppPage *page;      /* 若已映射，指向對應 suppPage     */
    struct thread *owner;       /* 擁有該 pagedir 的執行緒         */
    struct list_elem elem;      /* 串到全域 frame_table            */
    bool pinned;                /* true ⇒ 不得被驅逐               */
};
```

2. must pin the page before I/O operation: before any I/O operation (e.g. swap_in, swap_out, read/write user memory), vm_frame_pin() will be called to set the pinned bit of the frame to true to indicate that this page must not be evicted.

```
void
vm_frame_pin (void *kva)
{
    if (kva == NULL) return;

    /* 檢查當前線程是否已經持有鎖，如果是則直接操作 */
    bool already_holding_lock = lock_held_by_current_thread(&frame_lock);

    if (!already_holding_lock)
        lock_acquire (&frame_lock);

    /* 查找要 pin 的frame */
    struct list_elem *e;

    for (e = list_begin (&frame_table); e != list_end (&frame_table); e = list_next (e))
    {
        struct frame *fr = elem_to_frame (e);
        if (fr->kva == kva)
        {
            fr->pinned = true;
            break;
        }
```

3. unpin after I/O: After the I/O operation is finished, call vm_frame_unpin() to set the

pinned bit back to false, so that the page can be driven back to the evictable state.

```
/* 允許外部快速 pin / unpin （I/O 前後） */
Tabnine | Edit | Test | Explain | Document
void frame_pin   (struct frame *fr);
Tabnine | Edit | Test | Explain | Document
void frame_unpin (struct frame *fr);
      zxc2905617, last week • 第一次上傳
#endif /* vm/frame.h */
```

```
void
vm_frame_unpin (void *kva)
{
    if (kva == NULL) return;

    /* 檢查當前線程是否已經持有鎖，如果是則直接操作 */
    bool already_holding_lock = lock_held_by_current_thread(&frame_lock);

    if (!already_holding_lock)
        lock_acquire (&frame_lock);

    /* 查找要 unpin 的frame */
    struct list_elem *e;

    for (e = list_begin (&frame_table); e != list_end (&frame_table); e = list_next (e))
    {
        struct frame *fr = elem_to_frame (e);
        if (fr->kva == kva)
        {
            fr->pinned = false;
            break;
        }
```

4. Victim will skip pinned frame when selecting: replace the algorithm in the Clock page (select_victim()), if the pinned of the frame is true, then it will be skipped directly without considering.This ensures that pages in I/O will not be selected as victims by mistake.

```
select_victim (void)
    /* 從 clock_hand 開始繞，只要繞過的 elem 數 >= 表大小就保證看過全部 */
    for (size_t scanned = 0; scanned < table_sz * 2; scanned++)
    {
        if (clock_hand == list_end (&frame_table))
            clock_hand = list_begin (&frame_table);

        struct frame *fr = elem_to_frame (clock_hand);
        clock_hand = list_next (clock_hand);        /* hand 往前移 */

        if (fr->pinned)        /* 被pin住 => 跳過 */
            continue;

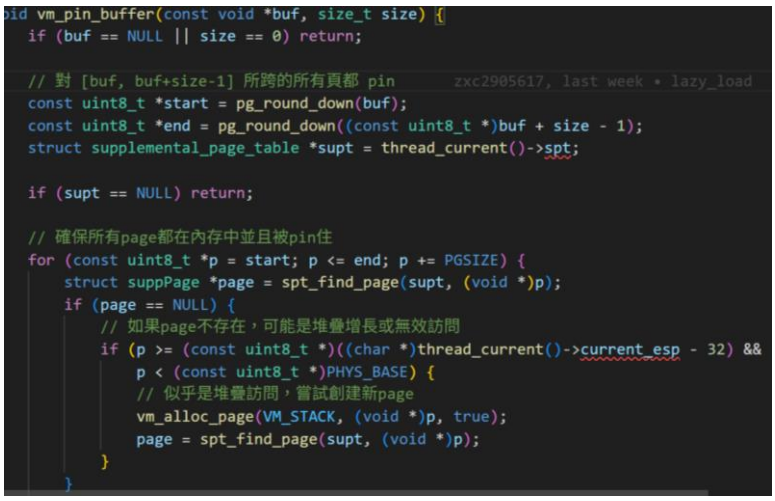
        /* accessed? 若被 access 就清 bit, 給第二次機會 */
        if (pagedir_is_accessed (fr->owner->pagedir, fr->page->va))
        {
            pagedir_set_accessed (fr->owner->pagedir, fr->page->va, false);
            continue;
        }
        /* un-pinned & accessed=0 → 正式選為 victim */
        return fr;
```

(d) Syscall-Time Page Access.

When the OS kernel handles a system call, it often directly accesses the buffer (e.g. read(), write(), exec(), etc.) passed in by the user program.If the page where the buffer is located has not yet been allocated or has been swapped out, a page fault will be triggered, which may result in a race, data loss, or even a deadlock if not properly designed.To ensure the accuracy and security of the kernel when accessing user memory, the system has implemented the following mechanism:

1. The System Call Handler will pin all pages of the user buffer in advance.

When entering system call handler (e.g. read/write), it will call vm_pin_buffer() to pin all the pages involved in the user buffer:



```
void vm_pin_buffer(const void *buf, size_t size) {
  if (buf == NULL || size == 0) return;

  // 對 [buf, buf+size-1] 所跨的所有頁都 pin      zxc2905617, last week • lazy_load
  const uint8_t *start = pg_round_down(buf);
  const uint8_t *end = pg_round_down((const uint8_t *)buf + size - 1);
  struct supplemental_page_table *supt = thread_current()->spt;

  if (supt == NULL) return;

  // 確保所有page都在內存中並且被pin住
  for (const uint8_t *p = start; p <= end; p += PGSIZE) {
      struct suppPage *page = spt_find_page(supt, (void *)p);
      if (page == NULL) {
          // 如果page不存在，可能是堆疊增長或無效訪問
          if (p >= (const uint8_t *)((char *)thread_current()->current_esp - 32) &&
              p < (const uint8_t *)PHYS_BASE) {
              // 似乎是堆疊訪問，嘗試創建新page
              vm_alloc_page(VM_STACK, (void *)p, true);
              page = spt_find_page(supt, (void *)p);
          }
      }
```

2. pin buffer process description

vm_pin_buffer() will count all user pages involved in the buffer (considering cross-page buffer), and call vm_pin_page() or vm_frame_pin() to set the pinned bit of each frame to true.

3. When finished, unpin to restore replaceable

After the system call handler finishes (or the critical operation finishes), it calls vm_unpin_buffer() to unpin these buffer pages:

```
void vm_unpin_buffer(const void *buf, size_t size) {
    if (buf == NULL || size == 0) return;

    // 對 [buf, buf+size-1] 所跨的所有頁都 unpin
    const uint8_t *start = pg_round_down(buf);
    const uint8_t *end = pg_round_down((const uint8_t *)buf + size - 1);
    struct supplemental_page_table *supt = thread_current()->spt;

    if (supt == NULL) return;

    for (const uint8_t *p = start; p <= end; p += PGSIZE) {
        vm_unpin_page(supt, (void *)p);
    }
}
```

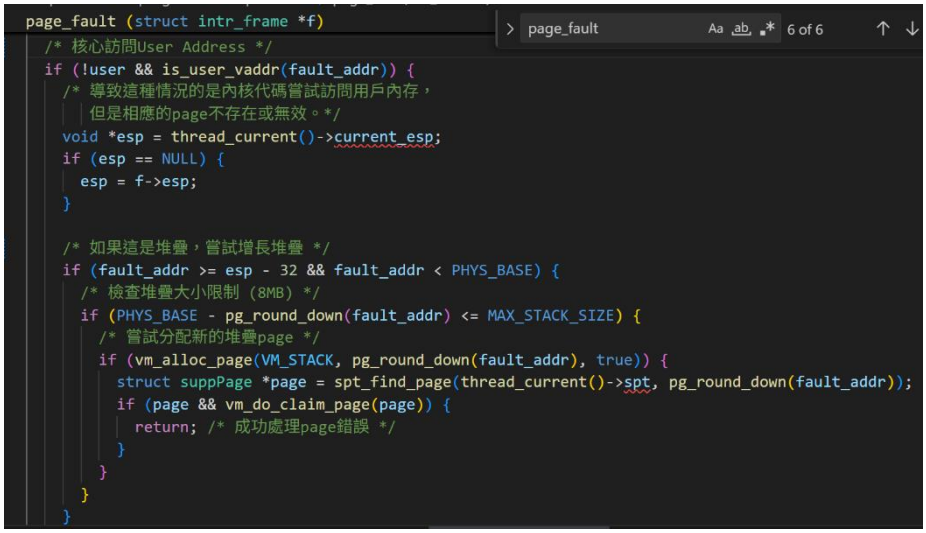
Flow Summary: Pinned Page I/O Protection Flow

1. System Call Handler Input Stage When the system call handler receives input (such as read/write file, I/O operation), it will first call vm_pin_buffer(buf, size) to protect the memory range for the specified buffer.
2. Memory Protection and Loading
   The system will check if the segment is loaded into physical memory.If any page is not yet loaded, the page will be claimed first and marked as pinned to avoid being driven by page replacement during I/O operations.
3. Perform core operations
   After a page has been fixed, perform the corresponding kernel I/O operations, such as file reading, file writing, or device interaction.
4. Unprotect after the operation is completed. When the operation is finished, vm_unpin_buffer(buf, size) is called to unpinned the segment and restore it to a candidate that can be replaced by the page replacement algorithm.

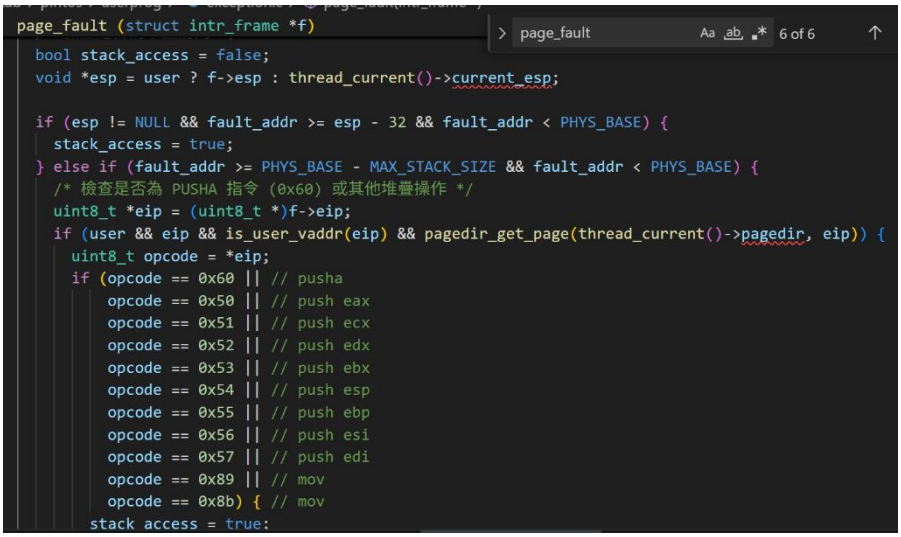
3. Stack Growth

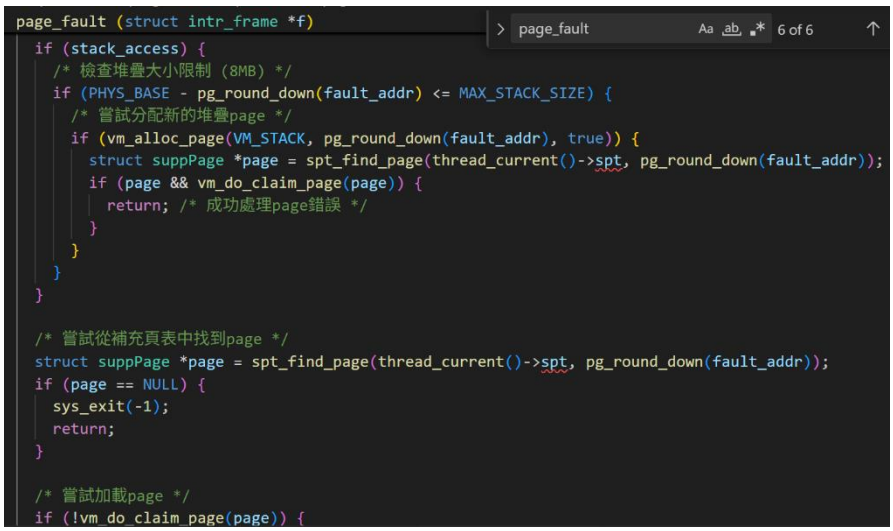
3.1 Stack-Fault Heuristic:

To support automatic stack growth, I have implemented a stack fault determination mechanism.When a page fault occurs at an invalid address, the system does not terminate the process immediately, but instead performs the following checks: in the page fault



Stack Growth judgment condition code and flow.The system determines if the fault address is close to an esp, or if it was triggered by a specific stack operation instruction, and performs maximum stack limits and page assignments.





When handling a page fault, the system first obtains the fault address and the current stack pointer (ESP).Depending on whether the access error is in user mode or kernel mode, the source of the stack pointer will be different: for user mode, the stack pointer will be taken from f->esp; for kernel mode, it may be taken from thread_current() ->current_esp.

Next, the system will determine whether the page fault is a legitimate stack growth behavior based on a number of conditions.

First, the system will check whether the fault address is close to the ESP.If the fault address falls within 32 bytes down from the ESP (i.e., fault_addr >= esp - 32), it is considered to be a possible legitimate stack expansion, such as a fault generated when a page triggered by a function call or push instruction has not been allocated.behavior is common in most compilers and operating systems.

Secondly, to prevent the program from growing its stack area indefinitely, the system will check whether the fault address falls within the maximum allowed stack area, which is usually set to 8MB, and in practice, it will do so by determining PHYS_BASE - pg_round_down(fault_addr) <= MAX_STACK_SIZEto limit the size of the stack.

Finally, as an advanced judgment mechanism (a bonus design), the system can also perform more precise identification based on the actual instruction that caused the page fault.If the instruction is an opcode related to stack operation (such as push eax, pusha, mov, etc.), then it can be further recognized as a reasonable demand for stack expansion.This type of design enhances the accuracy of heuristic by comparing the instruction contents, and improves the

system's ability to determine legitimate and illegitimate accesses.

Stacked Page Configuration and Processing Flow:

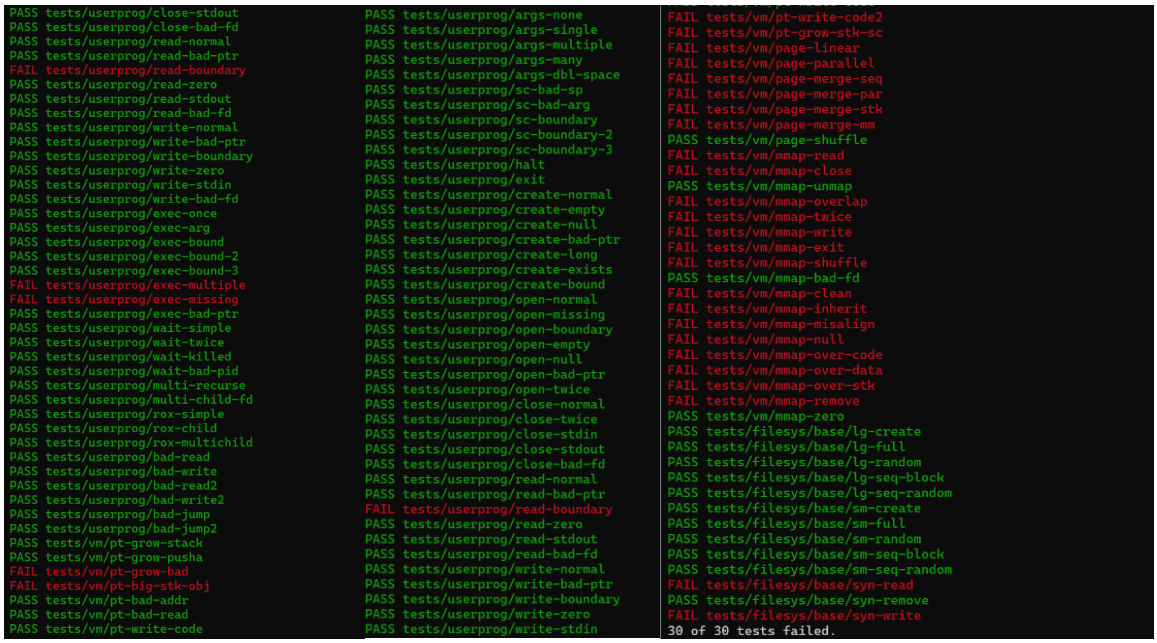
When the system determines that the page fault meets the legal stack growth conditions, it will take the following steps to configure the dynamic stack page.First, a new stack page is allocated at the alignment of the fault address by calling vm_alloc_page(VM_STACK, pg_round_down(fault_addr), true), and if the configuration is successful, the system will then perform a claim operation on the page, loading it into theThe system will then perform a claim operation on the page to load it into the physical memory to complete the whole memory expansion process.

However, if any of the configuration or claim steps fails, such as insufficient memory resources or abnormal structure creation, the system will directly terminate the process to prevent improper memory consumption or potential security vulnerabilities and to maintain the overall stability and security of the system.

Non-Stack Growth scenarios:

If the fault address does not meet any of the conditions for legitimate stack expansion (including distance from the ESP, whether it is within the maximum stack limit, whether it is a reasonable stack operation instruction, etc.), the system will consider the page fault as a "invalid access".In this case, in order to prevent malicious or wrong memory operation, the system will call sys_exit(-1) to terminate the corresponding process to ensure the safety and stability of the kernel and other user programs.

Test Result:



fail test case:

1. FAIL tests/userprog/read-boundary

This test checks the behavior when the read buffer just crosses the boundary between legal and illegal memory blocks (e.g., the first half of the buffer belongs to the legal block and the second half belongs to the illegal block).According to the specification, if any part of the buffer is illegal, the read operation should not be performed.A common mistake is to check only the starting address of the buffer, without checking one by one that the entire memory area covered by the buffer belongs to the legal area.

2. FAIL tests/userprog/exec-missing

The purpose of this quiz is to test the correct behavior of the system when exec system calls are used to execute a file that does not exist.There are two common errors: one is to try to execute the file without checking if it exists first; the other is to fail to return -1 to the parent process when the execution fails, resulting in the parent process getting the wrong result.

3. FAIL tests/userprog/exec-multiple

The quiz checks whether the parent process can correctly manage multiple child processes at the same time, including build, wait and recycle status.Failure is usually caused by

insufficient synchronization between parent and child, improper management of child list, or wrong transmission of exit status.To fix this, we need to add a synchronization mechanism to ensure that the child is initialized when the parent waits, and improve the parent's child list management and exit status transmission to avoid race condition and resource omission.

4. FAIL tests/filesys/base/syn-read

Common reasons for failure include: incomplete implementation of locks or synchronization mechanisms, such as the lack of locks on file operations, leading to conflicts when writing or deleting at the same time, and errors in reference counting, such as deleting a file without determining whether other processes are still in use, resulting in duplicate releases or data loss.In addition, the incorrect order of resource release is also a common problem. failure to properly handle the timing of file close and remove can lead to dangling pointer or double free. finally, if the cache is not properly flushed or write-back, the data only exists in the cache during synchronization and is not written back to the file, which can easily lead to a conflict.If the cache is not properly flushed or write-back, the data only exists in the cache during synchronization and is not written back to the file, which can easily cause data inconsistency.

5.FAIL tests/filesys/base/syn-write

Common reasons for failure include: incomplete implementation of locks or synchronization mechanisms, such as the lack of locks on file operations, leading to conflicts when writing or deleting at the same time, and errors in reference counting, such as deleting a file without determining whether other processes are still in use, resulting in duplicate releases or data loss.In addition, the incorrect order of resource release is also a common problem. failure to properly handle the timing of file close and remove can lead to dangling pointer or double free. finally, if the cache is not properly flushed or write-back, the data only exists in the cache during synchronization and is not written back to the file, which can easily lead to a conflict.If the cache is not properly flushed or write-back, the data only exists in the cache during synchronization and is not written back to the file, which can easily cause data inconsistency.

6. FAIL tests/vm/pt-grow-bad

The main purpose of this quiz is to test the logic of stack growth, when a fault address exceeds the legal stack growth range or does not meet the conditions, the system should not

allocate a new stack page for it, the common error is that the conditions are not designed strictly enough, which leads to the stack expansion when encountering illegal fault address, and thenThis can lead to security vulnerabilities or memory management errors.

7. FAIL tests/vm/pt-big-stk-obj

The purpose of this quiz is to test whether the system can allocate all the required stack pages continuously and correctly when the stack growth spans over multiple pages at a time, a common error is that only one page is allocated, which does not deal with the situation of allocating more than one page at a time, resulting in some of the memory not being able to be used correctly.

8. FAIL tests/vm/pt-bad-addr

The purpose of this quiz is to test the system's handling of illegal memory accesses, in which case the process should be terminated directly. Common errors include allowing illegal memory accesses, or failing to check the legality of the user pointer correctly, which can lead to system behavioral anomalies or security vulnerabilities.

9. FAIL tests/vm/pt-write-code2

A common reason for failure of this test is that the program tries to write to the code segment (read-only segment), but the system does not check the permissions of the read-only area.The correct behavior is to block the code segment when it is attempted to be written, otherwise the memory protection mechanism will be violated, leading to a potential system security problem.

10.FAIL tests/vm/pt-grow-stk-sc
This quiz mainly tests whether the stack growth checks are stringent.For example, whether the stack expansion exceeds the 1MB limit, or whether the fault address is close enough to the esp. If the conditions are insufficient, illegal stack expansion may occur.If the conditions are insufficient, illegal stack expansion will occur easily.When fixing the problem, it is necessary to ensure that the stack growth cannot exceed the specified limit, and the fault address must be close to esp before expansion can take place.

11.FAIL tests/vm/page-linear This is a test for linear access to a page.If the page replacement, lazy load, or claim frame mechanisms are not well implemented, it is easy to cause failures, and the focus of the fix is to check the logic of the page fault handler to make sure that the

load/eviction of the page is handled correctly.

12.FAIL tests/vm/page-parallel This test focuses on page faults and page replacements in multi-threaded or multi-process environments, where a common cause of failure is that the frame management is not properly synchronized, leading to a race condition; the fix is to make sure that there is a proper synchronization mechanism for the allocation and recycling of frames.

13.FAIL tests/vm/page-merge-seq

This test checks that page frames are correctly reclaimed when a page is re-released from a consecutive visit, that repeated releases are avoided, and that page table entries are correctly cleared, and that page table entries are correctly updated each time a page is released. The state of the page table and frame is correctly updated each time the page is released.

14.FAIL tests/vm/page-merge-par

A common error in page merge tests in multi-threaded environments is that the reference count is not handled correctly, or the frame is not delayed in release. To fix this, we need to use reference counting and make sure that only the last owner releases the frame.

15.FAIL tests/vm/page-merge-mm

This test measures the order and dirty page handling when merging or recycling after multiple memory-mapped file operations. Failure is usually due to wrong frame reclaim sequence or missing dirty page write-back, and it is important to make sure that mmap/unmap can correctly handle all page reclaims and dirty page write-backs.

16.FAIL tests/vm/page-merge-stk

Test if stack growth generates multiple pages, and then releases all of them to avoid duplicate releases or dirty page omissions. The key point of the fix is to make sure that the stack page release process is complete and correct.

17.FAIL tests/vm/page-shuffle

This test will repeatedly perform page allocation and recovery (shuffle), if the page table, frame or swap is not synchronized correctly, it is easy to fail. If the page table, frame or swap

are not synchronized correctly, it is easy to fail. When fixing this, the synchronization of page, frame and swap as well as the complete recycling of resources should be implemented.

18.FAIL tests/vm/map-unmap

mmap/unmap does not maintain the mapping table correctly, or unmmap does not release all pages, or dirty pages are not written back, it is easy to fail. When fixing, make sure that every mapping/unmapping can recover the page and write back the file correctly.

19.FAIL tests/vm/map-overlap

When two maps overlap, if the mapping table is not maintained, the permission is not checked, or there is a bug in the unmap, it is easy to generate errors. If the mapping table is not well maintained, the permissions are not checked, or there is a bug in unmmap, it is easy to generate an error.

20.FAIL tests/vm/mmap-unmap

When the mmap/unmap function is not fully implemented, the unmap may not release the page or the dirty page may not be written back to the file. When fixing this, it is necessary to manage the mapping table correctly, unmap handle dirty/clean page and release the resources.

21.FAIL tests/vm/mmap-unmap-twice

If you unmmap the same mapping twice without checking the repeated unmmap, it will result in double free. When fixing the unmmap, you need to check whether it has been unmmap or not, to avoid repeated release.

22.FAIL tests/vm/mmap-shuffle

This test will do mapping/unmapping/shuffle operation for many times, it is easy to release table or resources in wrong order, which will cause part of the page missed and not released. The key point of the fix is that the mmap/unmap action should correspond to all pages and release them correctly.

23.FAIL tests/vm/mmap-close

If the mapping still exists when the file is closed, or the unmmap is not written back to the

file in time, the test will fail. To fix this, unmmap must finish writing back all dirty pages before closing.

24.FAIL tests/vm/mmap-inherit

mmap content is not synchronized when fork or process inherits, or is released by mistake. Fix this by synchronizing the mapping table with the new process.

25. FAIL tests/vm/mmap-misalign

mapping/unmapping start address or length is not page-aligned, if it is not processed or mis-accepted, it will be wrong. The key point is to check all mapping must be page-aligned.

26.FAIL tests/vm/mmap-null

mmap NULL indicator should fail directly, but may allow mapping, resulting in an error. Make sure that the NULL pointer cannot be mmap when fixing it.

27.FAIL tests/vm/mmap-over-data

allow mmap to existing data/heap addresses will overlap, this case should fail. check if the mapping section is already occupied when fixing.

28.FAIL tests/vm/mmap-over-stk

If mmap to stack segment is not blocked, or unmmap releases stack page by mistake, it will fail. The main point of the fix is that mapping cannot cover the stack.

29.FAIL tests/vm/mmap-over-code

It is wrong to allow mmap to code segment, this behavior should be checked and blocked. When fixing this, mapping should avoid overlaying code segments.

30.FAIL tests/vm/mmap-remove

unmmap misses page table or frame cleanup. The main point of the fix is to remove all mapping, page table and frame resources when unmmap.